TEMPLE COLLEGE
WORKFORCE DEVELOPMENT

CHIEFOS

STUDENT REVIEW PACKET

Week 11

Branches, undoing things, GitHub + SSH, push, clone

Phase 2 — Building Real Apps · Weeks 7–12 · Modules 03–04

No D2L login required. This is the same lesson text posted in Brightspace for this week. Work the lessons in order on your own machine. Keep each checkpoint screenshot or pasted output. Bring it to the live session, or email it to your instructor, instead of submitting in D2L. Module quizzes stay in D2L and are not included here.

In this packet

1. 04.05 Branches
2. 04.06 Undoing Things
3. 04.07 GitHub Account and SSH Key
4. 04.08 Pushing to GitHub
5. 04.09 Cloning, Pulling, and Conflicts

Learning objective: Create and switch branches with `git switch`, merge a feature branch into `main` via both a fast-forward and a true merge commit, and delete merged branches with `git branch -d`.

Lesson 05: Branches

By the end, you will have created a feature branch, made commits on it, merged it back to `main`, and deleted it.

~25 minutes of typing

> A **branch** is a movable pointer to a commit. Switching branches updates your working tree to look like that commit. Branches are cheap, free, and the unit of "work in progress."

---

1. Confirm clean state.

```
cd ~/docker-course/fastapi-app
git status
```

2. List existing branches.

```
git branch
```

```
* main
```

(The `*` is the current branch.)

3. Create a new branch called `add-health-route`.

```
git branch add-health-route
```

(No output = success.)

4. List branches again.

```
git branch
```

```
  add-health-route
* main
```

(The branch exists, but you're still on `main`.)

5. Switch to the new branch.

```
git switch add-health-route
```

```
Switched to branch 'add-health-route'
```

(Older Git uses `git checkout <branch>` — both work.)

6. Confirm.

```
git branch
```

```
* add-health-route
  main
```

7. Look at the bottom-left of your VS Code window — it should now say `add-health-route` instead of `main`. VS Code reads `.git/HEAD` to show the current branch.

8. Quick shortcut: create + switch in one command. (We won't actually use this now — just know it exists.)

```
# git switch -c another-branch     # creates AND switches
# git checkout -b another-branch   # older equivalent
```

9. Now make a real change on this branch. Open `main.py` in VS Code. Above `app.include_router(items.router)`, add a `/health` route:

```


@app.get("/health")
def health():
    return {"status": "ok"}
```

10. Save.

11. Status.

```
git status
```

You should see `main.py` modified.

12. Commit it.

```
git add main.py
git commit -m "Add /health route"
```

13. See the log on this branch.

```
git log --oneline -5
```

The new commit is on top, on `add-health-route`.

14. Now switch back to `main`.

```
git switch main
```

15. Open `main.py` in VS Code. **The `/health` route is gone.** It only exists on the other branch. Switching branches changes the files on disk to match that branch's commit.

16. Confirm with the log.

```
git log --oneline -3
```

The "Add /health route" commit is missing — it's only on the other branch.

17. See **all** branches' commits in a graph.

```
git log --oneline --graph --all --decorate -10
```

You should see a graph where `add-health-route` has one extra commit beyond `main`.

18. Switch back to the feature branch.

```
git switch add-health-route
```

19. Open `main.py`. The `/health` route is back. Branches keep their own state.

20. Make another commit on this branch. Edit `main.py` and add a docstring under the `health` function:

```
    """Liveness probe — returns 200 if the app is up."""
```

21. Save, stage, commit.

```
git commit -am "Document /health route"
```

22. Look at the graph.

```
git log --oneline --graph --all --decorate -10
```

Two commits ahead of `main` now.

23. Now merge the feature branch back into `main`. First, switch to `main`.

```
git switch main
```

24. Merge.

```
git merge add-health-route
```

```
Updating abc1234..def5678
Fast-forward
 main.py | 5 +++++
 1 file changed, 5 insertions(+)
```

Read it. "Fast-forward" means `main` had no commits of its own since the branch diverged, so Git just moved the `main` pointer forward — no merge commit needed.

25. Confirm `main.py` now has the `/health` route.

```
grep -A 2 "def health" main.py
```

26. History on `main`.

```
git log --oneline -5
```

The `/health` commits are now on `main`.

27. Delete the merged feature branch — its commits live on `main` now.

```
git branch -d add-health-route
```

```
Deleted branch add-health-route (was abcdef0).
```

28. Confirm.

```
git branch
```

```
* main
```

29. Now feel a non-fast-forward merge. Create a new branch, make a commit on it, but **also** make a commit on `main` in between. This forces Git to create a real merge commit.

30. Create + switch.

```
git switch -c rename-hello
```

31. Edit `main.py`'s `read_root` to return `{"hello": "branched"}`. Save, commit.

```
git commit -am "Rename hello response to 'branched'"
```

32. Switch back to `main`.

```
git switch main
```

33. Make a different change. Edit `NOTES.md` and add a line `Branch demo`. Save, commit.

```
git commit -am "Note branch demo in NOTES.md"
```

34. Look at the graph — `main` and `rename-hello` have diverged.

```
git log --oneline --graph --all --decorate -10
```

35. Merge — this will be a true merge commit.

```
git merge rename-hello -m "Merge rename-hello into main"
```

```
Merge made by the 'ort' strategy.
 main.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

36. See the merge commit in the graph.

```
git log --oneline --graph --decorate -10
```

You should see the diamond shape: two parents converging on the merge commit.

37. Delete the merged branch.

```
git branch -d rename-hello
```

38. Open `main.py` and confirm it now says `{"hello": "branched"}`. Both changes (the rename and the note) are present.

39. Log it.

```
echo "Branches: created, committed, merged (FF + true merge)" >> ~/docker-course/notes/notes.txt
```

Watch out

- **Always `git status` and `git branch` before doing anything destructive.** Knowing where you are prevents 95% of mistakes.
- `git branch -d` only deletes a fully merged branch (safe). `git branch -D` force-deletes any branch (loses unmerged work).
- **You can't switch branches if you have uncommitted changes** that would conflict. Either commit them, or `git stash` (covered briefly later).
- **`git switch` is newer and clearer than `git checkout`.** Both work. The lessons use `switch` for branches and reserve `checkout` for older or specific patterns.

Checkpoint

```
git branch
```

Should show only `* main`.

```
git log --oneline --graph -8
```

Should show your merge commit (look for the asterisk graph showing two parents).

```
grep '"branched"' main.py
```

Should print the line with `branched`.

```
grep "Branch demo" NOTES.md
```

Should print the note.

Next up: 06-undoing-things.md — When you mess up

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Undo common mistakes by discarding edits with `git restore`, unstaging with `git restore --staged`, amending the last commit with `git commit --amend`, and removing a commit with `git reset`.

Lesson 06: Undoing Things

By the end, you will know how to discard unsaved changes, unstage files, fix the last commit message, and (carefully) reset a commit.

~20 minutes of typing

> Almost every "I broke Git" panic is one of four problems. Each has a one-liner fix. Today: drill all four.

---

1. Confirm clean state.

```
cd ~/docker-course/fastapi-app
git status
```

Case 1: Discard an unstaged edit

2. Make a change you don't want to keep. Edit `main.py` — change `{"hello": "branched"}` to `{"hello": "MISTAKE"}`. Save.

3. Confirm Git sees it.

```
git diff
```

You should see the bad change.

4. Discard it. `git restore` reverts the working file to whatever's in the latest commit.

```
git restore main.py
```

5. Confirm.

```
git diff
```

(No output — the change is gone.)

6. Open `main.py` in VS Code — should show `{"hello": "branched"}` again.

Case 2: Unstage a file (you `git add`-ed by mistake)

7. Make a change. Edit `NOTES.md` and add a line `oops`. Save.

8. Stage it.

```
git add NOTES.md
```

9. Status.

```
git status
```

You should see `NOTES.md` under "Changes to be committed."

10. Unstage it (the change stays in your working tree, just not staged).

```
git restore --staged NOTES.md
```

11. Status.

```
git status
```

`NOTES.md` should be back under "Changes not staged."

12. Now also discard the working-tree change.

```
git restore NOTES.md
```

13. Confirm clean.

```
git status
```

Case 3: Fix the last commit message (or add a forgotten file)

14. Make a real change worth committing. Edit `NOTES.md` to add `Undo drills`. Save.

```
git commit -am "fixt typo"
```

(Yes, the message is misspelled on purpose.)

15. Look at the bad message.

```
git log --oneline -1
```

```
abc1234 (HEAD -> main) fixt typo
```

16. Amend the message.

```
git commit --amend -m "Note undo drills in NOTES.md"
```

17. Confirm.

```
git log --oneline -1
```

```
def5678 (HEAD -> main) Note undo drills in NOTES.md
```

Notice the hash changed. Amending creates a new commit replacing the old one. **Don't amend commits you've already pushed** — it rewrites history and confuses anyone who pulled the original.

18. You can also amend to add a forgotten file. Edit `NOTES.md` again, add `another line`. Save.

```
git add NOTES.md
git commit --amend --no-edit
```

(`--no-edit` keeps the existing message unchanged.)

19. Confirm.

```
git log -1 --stat
```

The commit now contains both edits to `NOTES.md`.

Case 4: Undo the last commit entirely

20. Make a commit you want to undo. Edit `NOTES.md` and add `BAD COMMIT`. Save.

```
git commit -am "this commit shouldn't exist"
```

21. See it.

```
git log --oneline -3
```

22. Option A: keep the changes, undo only the commit. This is `--mixed` (the default).

```
git reset HEAD~1
```

The commit is gone but the file change is still on disk, unstaged.

23. Confirm.

```
git status
```

You should see `NOTES.md` modified.

```
git log --oneline -3
```

The bad commit is gone.

24. Discard the working-tree change.

```
git restore NOTES.md
```

Case 5: `git reset --hard` (the dangerous one)

25. This is the only Git command in this module that **loses uncommitted work**. Read carefully.

26. Make changes you'd want to keep. Edit `NOTES.md` to add `something important`. Save. **Don't commit.**

27. Suppose you got confused and ran (in real life) the wrong command. Here's what `--hard` does:

```
git reset --hard HEAD
```

```
HEAD is now at <hash> Note undo drills in NOTES.md
```

28. Open `NOTES.md`. `something important` **is gone.** `--hard` blew it away to match the latest commit.

29. Rule of thumb:

- `git restore <file>` — discard one file's working-tree changes (safe, clear scope)
- `git reset HEAD~1` — undo last commit, keep changes (safe-ish)
- `git reset --hard HEAD~1` — undo last commit AND lose the changes (**only use when you mean it**)

30. Some courses teach `git reflog` as a safety net for `--hard` mistakes. We won't drill it today — just know it exists. If you ever do `git reset --hard` on something important, `git reflog` may show the old commit hash so you can recover.

Recap drill

31. Make a clean commit to leave the repo tidy. Edit `NOTES.md` to add a line `End of lesson 6` (or just `End of lesson 6`). Save.

```
git commit -am "Mark end of lesson 6"
```

32. Confirm clean.

```
git status && git log --oneline -3
```

33. Log.

```
echo "Undo cycle drilled: restore, --staged, --amend, reset" >> ~/docker-course/notes/notes.txt
```

Watch out

- `git restore <file>` uses the same name as `git restore --staged <file>` — the `--staged` flag is what changes its meaning. Read the flag, not just the command.
- `git reset` **defaults to** `--mixed` (keep changes, unstage). `--soft` keeps changes staged. `--hard` discards changes. **Always type the flag explicitly so you know what you're getting.**
- **Never amend or reset commits you've pushed** to a remote. It rewrites history and forces a `--force` push, which is hostile to anyone else who pulled the original.

- `git reflog` is the rescue tool for "I lost a commit." Mention it when a student panics, walk them through it.

Checkpoint

```
git status
```

Should say `nothing to commit, working tree clean`.

```
git log --oneline | head -n 1
```

Should show your "Mark end of lesson 6" commit.

```
grep "End of lesson 6" NOTES.md
```

Should match.

Next up: 07-github-account-and-ssh.md — Connect to your GitHub account

> **Lab — do it for real.** Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Generate an ed25519 SSH key with `ssh-keygen`, register the public key in GitHub's settings, and verify passwordless authentication with `ssh -T`.

Lesson 07: GitHub Account and SSH

By the end, you will have a free GitHub account, an SSH key on your machine, and that key registered with GitHub so you can push without typing a password.

~20 minutes of typing + clicking

> **GitHub** is the largest Git host in the world — the same site most companies and open-source projects use. Your course repos will be **public**, on purpose: classmates can browse, clone, and review each other's work, exactly like real open source.

Step 1: create your GitHub account

1. Open your browser. Go to:

```
https://github.com/signup
```

(If you already have a GitHub account, sign in and skip to step 4.)

2. Use the **same email** you set in `git config --global user.email` (lesson 01). Mismatched emails create commits that don't link to your profile.

3. Pick a username carefully. It will be **public** — visible to classmates now and to future employers later. Something professional based on your name beats `xX_c0d3r_Xx`. Complete the email verification GitHub sends you.

4. Once logged in, look at the top-right of the page — your avatar should appear. Click it → **Settings** → **Emails** → confirm your email matches the one from lesson 01.

Step 2: generate an SSH key (if you don't have one)

5. Back in VS Code's integrated terminal. Check whether you already have a key.

```
ls ~/.ssh/id_ed25519.pub 2>/dev/null
```

If you see a path printed, you already have a key — **skip to Step 3**.

6. Generate one. The `-t ed25519` picks a modern key type. The `-C "..."` is just a label.

```
ssh-keygen -t ed25519 -C "you@example.com"
```

7. Three prompts:

- **"Enter file in which to save the key"** — press Enter to accept the default `~/.ssh/id_ed25519`.
- **"Enter passphrase"** — for this course, press Enter twice for no passphrase. (In real life, set one.)
- **"Enter same passphrase again"** — Enter again.

8. Output:

```
Your identification has been saved in /home/your-user/.ssh/id_ed25519
Your public key has been saved in /home/your-user/.ssh/id_ed25519.pub
The key fingerprint is: SHA256:...
```

9. Confirm both files exist.

```
ls -la ~/.ssh/id_ed25519*
```

You should see two files: `id_ed25519` (private) and `id_ed25519.pub` (public).

10. The private key never leaves your machine. Permissions should be `600` (owner read/write only). Verify.

```
ls -la ~/.ssh/id_ed25519
```

The permissions should look like `-rw-------`.

11. If they're not, fix them.

```
chmod 600 ~/.ssh/id_ed25519
```

Step 3: copy the public key

12. Print the **public** key (the `.pub` file).

```
cat ~/.ssh/id_ed25519.pub
```

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI... you@example.com
```

13. Select that entire line in your terminal and copy it (right-click → Copy, or Ctrl+Shift+C in most terminals).

Step 4: register the key on GitHub

14. In your browser, go to **github.com** → **top-right avatar** → **Settings** → **SSH and GPG keys**.

15. Click **New SSH key** (the green button).

16. Fill in:

- **Title:** something like `course-laptop`. (Just a label for you.)
- **Key type:** leave as **Authentication Key**.
- **Key:** paste the entire public key line from step 13.

17. Click **Add SSH key**. GitHub may ask you to re-enter your password to confirm. The key should appear in the list.

Step 5: test the SSH connection

18. Test from the terminal.

```
ssh -T git@github.com
```

19. First time, you'll see:

```
The authenticity of host 'github.com (140.82.x.x)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Type `yes` and press Enter. (Don't just press Enter — there's no default.)

20. Successful auth looks like:

```
Hi <username>! You've successfully authenticated, but GitHub does not provide shell
access.
```

That message means it worked. GitHub won't give you a shell, but it confirmed your identity.

21. If instead you see `Permission denied (publickey)`:

- Make sure you pasted the **public** key (`.pub` file), not the private one.
- Make sure you pasted the **whole line**, including `ssh-ed25519` at the start and the email at the end.
- Check the private key's permissions (`chmod 600 ~/.ssh/id_ed25519` — SSH refuses keys that other users can read).
- Run `ssh -vT git@github.com` to see verbose output.

22. Log it.

```
echo "GitHub SSH auth: working ($(ssh -T git@github.com 2>&1 | head -n 1))" >>
~/docker-course/notes/notes.txt
```

Watch out

- **Public vs private key.** You paste the `.pub` file's contents into GitHub. Never paste the private key (`id_ed25519` without `.pub`). The private key stays on your machine forever. If you accidentally pasted the private key anywhere, delete that key pair and generate a new one.
- **Wrong key permissions.** If the private key is readable by anyone else (`-rw-r--r--`), SSH silently refuses to use it. `chmod 600 ~/.ssh/id_ed25519` fixes it.
- **Some classroom networks block outbound port 22.** If `ssh -T` hangs, GitHub offers SSH over port 443: test with `ssh -T -p 443 git@ssh.github.com`. Ask the instructor before setting that up permanently.
- **Email mismatch.** If your GitHub email doesn't match `git config --global user.email`, GitHub won't link your commits to your profile. Fix one or the other to match.

Checkpoint

```
ls ~/.ssh/id_ed25519.pub
```

Should print the path.

```
ssh -T git@github.com
```

Should print `Hi <username>! You've successfully authenticated, but GitHub does not provide shell access.`

In the GitHub web UI, your SSH key should appear at **Settings** → **SSH and GPG keys** with the title you gave it.

Next up: 08-pushing-to-github.md — Push your repo to GitHub

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Connect a local repository to a GitHub remote with `git remote add origin`, push it with `git push -u origin main`, and round-trip commits between the terminal and the web UI with `git push` and `git pull`.

Lesson 08: Pushing to GitHub

By the end, your local FastAPI repo is mirrored in a public repo on GitHub, and you've made a commit on each side and synced them.

~15 minutes of typing + clicking

Step 1: create the empty repo on GitHub

1. In your browser, go to github.com. Click the **"+" icon** in the top-right → **New repository**.

2. Fill in:

- **Owner:** your username.
- **Repository name:** `fastapi-app` (matches your local folder name).
- **Visibility: Public.** Your classmates will browse and clone each other's repos in lessons 09–10 — that's the point.
- **Initialize this repository:** leave **everything unchecked** — no README, no `.gitignore`, no license. We're pushing an existing local repo — initializing on the server would create a conflict.

3. Click **Create repository**.

4. GitHub drops you on the quick-setup page. Find the section **"…or push an existing repository from the command line"**:

```
git remote add origin git@github.com:<username>/fastapi-app.git
git branch -M main
git push -u origin main
```

Make sure the **SSH** tab is selected (not HTTPS) — the URL must start with `git@github.com:`.

> **Fallback — Module 03 project broken or missing?** Don't rebuild it in class. Open the instructor's template repo at `github.com/<INSTRUCTOR-GITHUB-USERNAME>/course-items-api`, click **Use this template → Create a new repository**, name it `fastapi-app`, keep it **Public**, then clone it:
>
> ```
> git clone git@github.com:<username>/fastapi-app.git ~/docker-course/fastapi-app
> ```
>
> Cloning sets up `origin` for you, so skip Steps 2 and 3 below and continue at **Step 4: round-trip a commit**.

Step 2: connect local to remote

5. Back in the terminal, in your project.

```
cd ~/docker-course/fastapi-app
```

6. Confirm you have no remote yet.

```
git remote
```

(No output.)

7. Add the GitHub repo as the `origin` remote.

```
git remote add origin git@github.com:<username>/fastapi-app.git
```

(Substitute your real username from step 4.)

8. Confirm.

```
git remote -v
```

```
origin  git@github.com:<username>/fastapi-app.git (fetch)
origin  git@github.com:<username>/fastapi-app.git (push)
```

Step 3: push

9. Push your `main` branch and set it as the upstream tracking branch (the `-u` does that).

```
git push -u origin main
```

You should see:

```
Enumerating objects: ... done.
Counting objects: 100% done.
...
To github.com:<username>/fastapi-app.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

10. Refresh the GitHub repo page in your browser. **Your code is there.** All your files, all your commits, all your branches' commits that ended up on `main`.

11. Click on a few files to confirm. Click the **"N commits"** link (top-right of the file list, with a clock icon) to see your history.

Step 4: round-trip a commit

12. Make a change locally. Edit `NOTES.md` and add `Pushed to GitHub today`. Save.

```
git commit -am "Note GitHub push"
```

13. Push.

```
git push
```

(No `-u` this time — `main` already tracks `origin/main`.)

```
To github.com:<username>/fastapi-app.git
   abc1234..def5678  main -> main
```

14. Refresh GitHub. The new commit is at the top of the history. Click the commit message to see its diff rendered in the browser — green added lines, red removed lines, same as `git show`.

Step 5: edit on GitHub, pull locally

15. On GitHub, navigate to `NOTES.md`. Click the **pencil/edit icon** at the top-right of the file view.

16. Add a line at the bottom: `Edited via GitHub web UI`.

17. Click **Commit changes...** (green button, top-right). Set the commit message to `Edit NOTES via web`, keep **Commit directly to the `main` branch** selected, and click **Commit changes** (we'll do PRs in lesson 10).

18. Refresh the commits page — you should see the new commit.

19. Back in the terminal. **Don't edit anything locally.** Pull from GitHub.

```
git pull
```

```
remote: Enumerating objects: ... done.
...
Updating ...
Fast-forward
 NOTES.md | 1 +
 1 file changed, 1 insertion(+)
```

20. Open `NOTES.md` in VS Code. The line `Edited via GitHub web UI` should now be there.

21. Confirm with log.

```
git log --oneline -3
```

Step 6: see the SSH-vs-HTTPS distinction

22. Look at the remote URL.

```
git remote get-url origin
```

```
git@github.com:<username>/fastapi-app.git
```

SSH form — used for pushes/pulls without typing a password (your SSH key authenticates).

23. The HTTPS form would look like `https://github.com/<username>/fastapi-app.git`. GitHub doesn't accept your account password for HTTPS pushes — you'd need a personal access token. SSH is simpler once set up — that's why we're using it.

24. Log it.

```
echo "GitHub repo + first push: $(git remote get-url origin)" >> ~/docker-course/notes/notes.txt
```

Watch out

- **Your repo is public.** Anyone on the internet can read it. That's fine for course exercises — but never commit secrets. Your `.env` is in `.gitignore` from lesson 02 for exactly this reason. Check `git status` before every push.
- **"failed to push some refs"** means the remote has commits you don't have. Run `git pull` first, resolve any conflicts (lesson 09), then push again.
- **Don't initialize the GitHub repo with a README** if you have an existing local repo. The histories won't match and you'll get rejected pushes.
- **`-u` is only needed the first time** for a new branch. After that, `git push` knows where to go.
- **The remote name `origin` is convention, not magic.** You could call it anything. `origin` is what most tools assume.

Checkpoint

```
git remote -v
```

Should show `origin` with a `git@github.com:` SSH URL.

```
git push --dry-run
```

Should print `Everything up-to-date` (or what would be pushed).

On GitHub, the **"N commits"** count on your repo page should match `git log --oneline | wc -l` locally.

Next up: 09-cloning-and-pulling.md — The other side of collaboration

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Clone a repository from GitHub, sync changes between two working copies with `git pull`, and resolve a merge conflict by editing out the conflict markers, staging the file, and committing the merge.

Lesson 09: Cloning, Pulling, and Conflicts

By the end, you will have cloned your repo to a second directory, made conflicting commits in both, resolved the conflict — and cloned a classmate's public repo.

~20 minutes of typing

> Cloning is how teammates get a copy of the repo. Pulling is how they keep it in sync. When two people change the same lines, you get a **merge conflict** — and you learn to read the markers in the file and pick a side.

Part 1: clone

1. Move out of the project to your course directory.

```
cd ~/docker-course
```

2. Clone your own GitHub repo into a new directory called `fastapi-app-clone`.

```
git clone git@github.com:<username>/fastapi-app.git fastapi-app-clone
```

```
Cloning into 'fastapi-app-clone'...
remote: Enumerating objects: ... done.
Receiving objects: 100% done.
```

3. Two folders now.

```
ls ~/docker-course/ | grep fastapi-app
```

```
fastapi-app
fastapi-app-clone
```

4. Look at the clone.

```
cd fastapi-app-clone
git log --oneline -3
```

Same history as your original.

5. Confirm the remote was set automatically.

```
git remote -v
```

```
origin  git@github.com:<username>/fastapi-app.git (fetch)
origin  git@github.com:<username>/fastapi-app.git (push)
```

Cloning sets up `origin` for you. No `git remote add` needed.

Part 2: simulate a teammate

6. Pretend the clone is your teammate. Make a change there. Edit `NOTES.md` (in `fastapi-app-clone`) and add a line at the bottom: `Teammate: hello from the clone`. Save.

7. Commit and push.

```
git commit -am "Teammate adds hello note"
git push
```

8. Now switch back to your original.

```
cd ~/docker-course/fastapi-app
```

9. Pull the teammate's change.

```
git pull
```

```
remote: Enumerating objects: ... done.
...
Updating ...
Fast-forward
 NOTES.md | 1 +
 1 file changed, 1 insertion(+)
```

10. Confirm.

```
tail -n 3 NOTES.md
```

The teammate's line is now there.

11. This is the **happy-path collaboration loop**: edit, commit, push from one side; pull from the other.

Part 3: cause a conflict

12. In your **original** (`~/docker-course/fastapi-app`), edit `NOTES.md` and change the `Teammate: hello from the clone` line to:

```
Teammate (original wins): hello from the clone
```

Save. Commit:

```
git commit -am "Edit teammate note from original"
```

13. Don't push yet. Switch to the clone:

```
cd ~/docker-course/fastapi-app-clone
```

14. Edit the **same line** in `NOTES.md` to:

```
Teammate (clone wins): hello from the clone
```

Save. Commit:

```
git commit -am "Edit teammate note from clone"
```

15. Push from the clone.

```
git push
```

That works — the clone is ahead of the remote.

16. Now back in the original. Try to push.

```
cd ~/docker-course/fastapi-app
git push
```

```
To github.com:<username>/fastapi-app.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to ...
hint: Updates were rejected because the remote contains work that you do
hint: not have locally.
```

Read the error. The remote has the clone's commit, which your original doesn't have. Git won't let you overwrite it.

17. Pull first.

```
git pull
```

```
Auto-merging NOTES.md
CONFLICT (content): Merge conflict in NOTES.md
Automatic merge failed; fix conflicts and then commit the result.
```

Conflict.

18. Status.

```
git status
```

```
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  ...
Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   NOTES.md
```

Part 4: resolve the conflict

19. Open `NOTES.md` in VS Code. You'll see conflict markers like:

```
<<<<<<< HEAD
Teammate (original wins): hello from the clone
=======
Teammate (clone wins): hello from the clone
>>>>>>> abc1234... (Edit teammate note from clone)
```

20. Read it:

- Everything between `<<<<<<< HEAD` and `=======` is **your** version (the original side).
- Everything between `=======` and `>>>>>>>` is **theirs** (the clone's version).

21. Pick a resolution. For this drill, hand-merge to one final line. Replace the entire conflict block with:

```
Teammate (merged): hello from the clone
```

22. Save. Confirm no markers remain.

```
grep -E "^<<<|^===|^>>>" NOTES.md
```

(No output = clean.)

23. Stage the resolved file.

```
git add NOTES.md
```

24. Status.

```
git status
```

```
All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)
```

25. Commit the merge.

```
git commit
```

Git opens an editor with a default message like `Merge branch 'main' of github.com:<username>/fastapi-app`. Save and exit. (In nano: Ctrl+O Enter, then Ctrl+X.)

26. Push.

```
git push
```

Now succeeds.

27. Look at the graph.

```
git log --oneline --graph -5
```

You should see the diamond shape of the merge commit.

28. Pull on the clone to get the merge.

```
cd ~/docker-course/fastapi-app-clone
git pull
```

29. Confirm both directories agree.

```
diff ~/docker-course/fastapi-app/NOTES.md ~/docker-course/fastapi-app-
clone/NOTES.md
```

(No output = identical.)

Part 5: clone a classmate's repo (cross-sharing)

Everyone's course repos are **public** — that's deliberate. You can read anyone's code; you can only push to your own.

30. Get a classmate's GitHub username (from the board or your instructor). Clone their repo over HTTPS — no SSH key is needed just to **read** a public repo.

```
cd ~/docker-course
git clone https://github.com/<classmate>/fastapi-app.git classmate-fastapi-app
```

```
Cloning into 'classmate-fastapi-app'...
remote: Enumerating objects: ... done.
Receiving objects: 100% done.
```

31. Browse it — their files, their history, their name as author.

```
cd classmate-fastapi-app
ls
git log --oneline -5
git log -1 --format="%an <%ae>"
```

Their commits, not yours. You just cloned a stranger's-eye view of a project — the same way you'd clone any open-source repo.

32. Now prove you can read but not write. Make a throwaway commit and try to push.

```
echo "visitor note" >> NOTES.md
git commit -am "Try pushing to a classmate's repo"
git push
```

If Git prompts `Username for 'https://github.com':`, that's already your answer — you have no write credentials for this repo. Press Ctrl+C. If your machine has GitHub credentials stored, you'll see the refusal directly:

```
remote: Permission to <classmate>/fastapi-app.git denied to <username>.
fatal: unable to access 'https://github.com/<classmate>/fastapi-app.git/': The
requested URL returned error: 403
```

Either way, read the outcome. Public means read-for-everyone, not write-for-everyone. Proposing changes to a repo you can't push to is what forks and pull requests are for — lesson 10.

33. Undo the throwaway commit (lesson 06's dangerous command, used on purpose — the commit is junk).

```
git reset --hard HEAD~1
```

34. Log it.

```
echo "Conflict resolved end-to-end; classmate repo cloned read-only" >> ~/docker-
course/notes/notes.txt
```

Watch out

- **Never commit a file with `<<<<<<<` markers.** VS Code will highlight them in red. They're for **you** to resolve, not to commit.
- **`git status` during a merge** is your friend. It tells you exactly which files still have unresolved conflicts.

- `git merge --abort` undoes a half-finished merge if you panic. You'll be back to where you started before `git pull`.
- **The clone is independent.** You committed in both directories — Git couldn't know one was "the original" and the other "the copy." They're peers.
- **Read is not write.** Anyone can clone a public repo; only the owner (and invited collaborators) can push. Don't be alarmed by the 403 in Part 5 — it's GitHub working as designed.

Checkpoint

```
cd ~/docker-course/fastapi-app
git status
```

Should be clean.

```
git log --oneline --graph -5
```

Should show your merge commit.

```
diff ~/docker-course/fastapi-app/NOTES.md ~/docker-course/fastapi-app-
clone/NOTES.md
```

Should produce no output.

```
ls ~/docker-course/classmate-fastapi-app/main.py
```

Should print the path — your classmate's project, cloned locally.

```
cd ~/docker-course/classmate-fastapi-app && git status
```

Should say `nothing to commit, working tree clean` (the throwaway commit is gone).

On GitHub, your repo's commit history should show the merge commit at the top.

Next up: 10-pull-requests.md — The team workflow

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.